

國立中正大學資訊工程所

碩士論文

Graduate Institute of Computer Science and Information Engineering

National Chung Cheng University

Master's Thesis

基於賽道記憶體之基因定序精確樣式比對演算法加速架

構

An Acceleration Architecture for Genome Sequencing Exact

Pattern Matching Algorithm Based on Racetrack Memory

研究生：劉季恆

Student: Ji-Heng Liu

指導教授：梁郁珮 博士

Advisor: Yu-Pei Liang, Ph.D.

中華民國 一一四年七月

July, 2025

中文摘要

隨著以資料為導向的應用不斷發展，例如即時資料搜尋、機器學習以及基因定序等，傳統的 von Neumann 架構在面對大量資料移動時的效能瓶頸日益顯著，進一步加劇了所謂的「memory wall problem」。為了解決這一問題，Processing-In-Memory (PIM) 架構應運而生，其通過在儲存裝置內進行計算來減少資料移動所需的時間和能耗。在眾多新興的非揮發性記憶體技術中，Racetrack Memory (RTM) 憑藉其高記憶體密度、優異的耐用性以及媲美 SRAM 的存取速度，成為 PIM 架構中備受關注的選擇。

本論文聚焦於基因定序中的 FM-index 搜尋，搜尋中的 symbol counting 的過程中會因大量的記憶體讀取需求導致 memory wall problem 的產生。針對此問題，本研究提出了 RTM Bucket-Size Processing-In-Memory (RT-BPIM) 架構，透過 RTM 的 Transverse Read (TR) 操作實現高效的 PIM 運算，並設計出針對 RTM shift 操作額外成本的改良機制，以減少存取成本。

實驗結果顯示，baseline 相比，RT-BPIM 能將 shift 操作的延遲降低 67%，整體延遲改善 75%，顯著提升 FM-index 搜尋的效率。同時，額外用於 PIM 的儲存空間與 baseline 相比只增加了 3.3%。

關鍵字：賽道記憶體、記憶體內運算

Abstract

With the rapid growth of data-driven applications such as real-time search, machine learning, and genome sequencing, the traditional von Neumann architecture faces increasing performance bottlenecks due to extensive data movement, exacerbating the "memory wall problem." Processing-In-Memory (PIM) emerges as a promising solution by enabling computations directly within memory, thereby reducing data transfer latency and energy consumption. Among emerging non-volatile memory technologies, Racetrack Memory (RTM) stands out due to its high density, exceptional durability, and SRAM-like access latency, making it a compelling choice for PIM architectures.

This study focuses on FM-index searches in genome sequencing, where the symbol counting process suffers from significant memory access overhead. To address this, we propose the RTM Bucket-Size Processing-In-Memory (RT-BPIM) architecture, leveraging RTM's Transverse Read (TR) operation for efficient PIM computations and introducing an optimized mechanism to reduce the overhead caused by RTM's shift operations.

Experimental results show that RT-BPIM reduces shift operation latency by 67% and overall latency by 75% compared to the baseline, significantly improving FM-index search efficiency. Furthermore, the additional storage required for PIM operations increases by only 3.3%.

Keywords: Racetrack Memory, Processing-In-Memory (PIM)

Table of contents

1. Introduction.....	1
2. Background and Related Works.....	3
2.1 Background.....	3
2.1.1 Burrow-Wheeler transform.....	3
2.1.2 FM-index.....	4
2.1.3 Racetrack Memory.....	7
2.1.4 RTM-base Main Memory Architecture.....	9
2.1.5 Transverse Read.....	10
2.2 Related Works.....	12
2.2.1 Processing In Memory.....	12
2.2.2 Pattern Matching with PIM.....	15
3. Motivation.....	18
4. RT-BPIM.....	20
4.1 Overview.....	20
4.2 Query of FM-index and Counting algorithm.....	22
4.3 Shift Optimization Design.....	24
4.4 RTM-base PIM Counting.....	25
4.4.1 Details of PIM DBC.....	27
4.4.2 In Memory symbol counting.....	28
5. Experiment and Evaluation.....	30
5.1 Experiment Setup.....	30
5.1.1 Simulation and Configuration.....	30
5.1.2 Dataset and Trace Generate.....	32
5.2 Experiment Evaluation.....	32

5.2.1 Latency Evaluation for Symbol Counting Operations.....	32
5.2.2 Energy Consumption Analysis.....	36
5.2.3 PIM DBC Overhead Analysis with Varying Bucket Sizes	38
6. Conclusion	39
REFERENCE.....	40



List of Figures

Fig. 1 : Transformation process of Burrows-Wheeler Transform.....	3
Fig. 2 : Relationship between bwt and suffix array	4
Fig. 3 : Auxiliary structures for FM-index.....	5
Fig. 4 : Anatomy of a RTM nanowire.....	8
Fig. 5 : Data Mapping on RTM.....	9
Fig. 6 : Transverse read example	10
Fig. 7 : Overview of the PIM enabled DBC extensions	14
Fig. 8 : ReRAM-based Hamming Distance Unit.....	16
Fig. 9 : RTM Main Memory Architecture.....	20
Fig. 10 : Storage Optimized FM-index	22
Fig. 11 : PIM DBC and TR Counting	26
Fig. 12 : Latency of symbol counting with bucket size 256	33
Fig. 13 : Latency of symbol counting with bucket size 512	33
Fig. 14 : Energy consumption of symbol counting with bucket size 256.....	36
Fig. 15 : Energy consumption of symbol counting with bucket size 512.....	37

List of Tables

Table 1 : Comparison of RTMs with other memory technologies.....	19
Table 2 : Latency and Energy consumption.....	31
Table 3 : Overhead Increase (%) of PIM DBC vs. Non-PIM Across Bucket Sizes	38



1. Introduction

With the rapid development of data-driven applications such as real-time data search, machine learning, and genome sequencing, these applications often involve significant data movement. This poses substantial challenges for traditional von Neumann systems. To address the issue of large-scale data movement, efforts have been made to enhance memory density and improve access speeds in storage devices. However, these attempts have not effectively bridged the performance gap between memory access and processor computation speeds, a phenomenon commonly referred to as the "memory wall problem." To mitigate this issue, a novel data processing paradigm has been proposed. Instead of transferring data from storage devices to processors for computation, this approach performs simple computations directly within the storage device by leveraging its intrinsic properties. This method, known as Processing-In-Memory (PIM), has been explored in prior research, including the use of DRAM for bulk-bitwise operations. Furthermore, emerging non-volatile memories (NVMs) [7][8], owing to their non-volatility, high memory density, and competitive access speeds, have also been applied in PIM architectures in recent years. Among these, Racetrack Memory (RTM) [6] has garnered significant attention. In addition to the aforementioned advantages, RTM exhibits superior endurance, higher memory density, and SRAM-like access speeds compared to other emerging NVMs [10], making it highly suitable for PIM architectures.

Among the aforementioned applications, genome sequencing has experienced unprecedented advancements in sequencing technologies. The progress of genome sequencing machines has surpassed Moore's Law, leading to challenges in storing the resulting massive amounts of sequence data and analyzing it at a pace that keeps up with its generation. To address these issues, a data transformation technique called

Burrows-Wheeler Transform (BWT) [1] has been proposed. BWT reduces storage demands by compressing the original data and supports precise pattern matching via a structure known as FM-index [4]. Precise pattern matching is a crucial step in genome sequencing. However, the FM-index search involves a symbol counting process that demands extensive memory reads, exacerbating the memory wall problem.

To address this problem, previous research [5] has employed ReRAM for PIM-based FM-index operations. Nevertheless, ReRAM suffers from endurance and area constraints. Compared to ReRAM, RTM demonstrates better performance in these aspects. Moreover, RTM features a unique operation called Transverse Read (TR) [16], which is particularly well-suited for PIM computations and has been widely utilized in prior studies [13].

This paper introduces RTM Bucket-Size Processing-In-Memory (RT-BPIM), an RTM-based architecture designed for PIM processing of FM-index searches. Due to the inherent characteristics of RTM, accessing data requires a shift operation, which incurs additional access costs compared to storage devices capable of random access. The proposed RT-BPIM architecture not only processes the symbol counting in FM-index searches using PIM but also mitigates the additional access costs arising from RTM's shift operations through an optimized design.

2. Background and Related Works

2.1 Background

2.1.1 Burrow-Wheeler transform

The Burrows-Wheeler Transform (BWT) [1], invented by Michael Burrows and David Wheeler in 1994, reorders the characters of a string without altering its composition. When a string undergoes the BWT, repeated substrings in the original string are transformed into consecutive repeated segments in the output. This property makes the BWT particularly suitable for data compression techniques. The BWT transformation process consists of two main steps: 1) Rotation and 2) Sorting, as illustrated in Fig. 1.

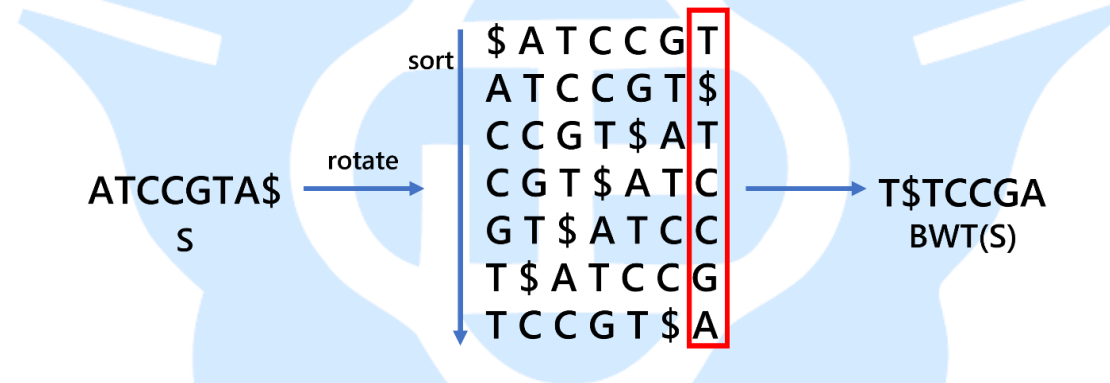


Fig. 1 : Transformation process of Burrows-Wheeler Transform

Consider the string S : $\text{ATCCGTA\$}$, where the symbol "\$" denotes the end-of-string marker. In the Rotation step, all possible rotations of the string are generated. Subsequently, in the Sorting step, these rotations are ordered lexicographically. The result of the transformation is derived by extracting the last column of the matrix formed by the sorted rotations. However, this method has a spatial complexity of $O(n^2)$, where n is the length of S , due to the requirement of generating all possible rotations. In fact, the transformation process shares similarities with the suffix array [3],

as shown in Fig. 2. A suffix array orders all suffixes of a string lexicographically. By observing the last column of the BWT output, it can be noted that these characters correspond to the preceding characters of each suffix in the suffix array. Therefore, the BWT result can be efficiently generated using the following formula (Eq. 1). Since some efficient algorithms, such as the SA-IS algorithm, can construct the suffix array in $O(n^2)$ time complexity, the spatial complexity of the BWT generation process can be reduced from $O(n^2)$ to $O(n)$.

	i	SA	
\$ ATCCGT	0	6	\$
ATCCGT\$	1	0	ATCCGT\$
CCGT\$AT	2	2	CCGT\$
CGT\$ATC	3	3	CGT\$
GT\$ATCC	4	4	GT\$
T\$ATCCG	5	5	T\$
TCCGT\$A	6	1	TCCGT\$

Fig. 2 : Relationship between bwt and suffix array

$$BWT[i] = \begin{cases} S[SA[i] - 1], & SA[i] > 0 \\ \$, & SA[i] = 0 \end{cases} \quad \text{Eq.(1)}$$

2.1.2 FM-index

The Ferragina-Manzini Index (FM-index) [4] is a compressed full-text substring index based on the Burrows-Wheeler Transform (BWT) [1]. Invented by Paolo Ferragina and Giovanni Manzini, this specialized index enables efficient pattern matching while maintaining a high compression rate. It leverages the properties of the suffix array and achieves fast lookups by calculating the rank and cumulative frequency of characters in the BWT. The FM-index supports substring searching and position restoration with high spatial efficiency, making it suitable for handling large-scale

textual data. It is widely applied in data compression and genomic sequence analysis, representing a further optimization of suffix-based structures. As previously discussed, to perform exact pattern matching with the FM-index, two auxiliary data structures, $\text{count}[]$ and $\text{occ}[]$, are required. The $\text{count}[]$ array stores the lexicographically sorted positions of each character's first occurrence, while the $\text{occ}[]$ array records the cumulative frequency of each character before each BWT index. Using S as an example, the results of these auxiliary structures are shown in Fig. 3.

Occ(c,i)				
i	A	C	G	T
0	0	0	0	0
1	0	0	0	1
2	0	0	0	1
3	0	0	0	2
4	0	1	0	2
5	0	2	0	2
6	0	2	1	2
7	1	2	1	1

Count(c)			
A	C	G	T
1	2	4	5

Fig. 3 : Auxiliary structures for FM-index

Both of these data structures can be precomputed. During exact pattern matching with the FM-index, a method called backward search [4] is employed, as demonstrated in Algorithm 1.

Algorithm 1 Backward Search (Q)

```

1: Input: Q (Query sequence)
2: Output: Matching interval or termination if no match is found
3: low  $\leftarrow$  0
4: high  $\leftarrow$  len(occ) - 1
5: for i  $\leftarrow$  len(Q) - 1 to 0 do
6:   low  $\leftarrow$  count[Q[i]] + occ(Q[i],low)
7:   high  $\leftarrow$  count[Q[i]] + occ(Q[i],high)
8:   if high < low then
9:     return
10:  end if
11: end for

```

The variables `low` and `high` in the two frequently executed lines of code are the core of the backward search process. The range defined by `low` and `high` represents the segment of the suffix array with the same prefix. `low` indicates the position of the suffix array where the first successful match of the prefix occurs. The count array specifies the starting point for the current search character range, but it only accounts for the current character and does not consider previously matched prefixes.

To address this, it is necessary to skip over irrelevant indices within the character range. This is achieved by leveraging a property of the FM-index: the characters in the first and last rows of the matrix (as shown in Fig. 1) appear in the same order. Thus, if a candidate character index in the first row has previously appeared before the current search character in the previous prefix, it implies that these characters, when rotated, do not correspond to the previous search characters. As a result, these characters must be skipped, which explains the addition of `occ` after `count` in the algorithm. `high`, on the other hand, represents the position immediately after the last successful match. At the start of the search, `low` and `high` are initialized to the minimum and maximum indices of `occ`, respectively. The calculation of `low` involves taking the original `low` and the target character as inputs, then computing the count at the index of `low` plus the `occ` value for the target character at the index of `low`. The process iterates from the last character of the target pattern to the first character. The calculation for `high` is similar to that of `low`. If `low` exceeds `high` during the search, it indicates that no match is found. Otherwise, the search is successful, and the occurrence count can be determined based on `low` and `high`. Additionally, the suffix array can be used to locate the positions of the matched substrings. For example, consider the transformed string `S:ATCCGTA$` and the query pattern `Q:TCC`. Before the search begins, `low` and `high` are initialized to 0 and 7, respectively. After the first iteration, `low` and `high` are updated to 2 and 4, indicating that the prefix starts at index 2 in the BWT and ends before index 4. After

three iterations, the final values of low and high are 6 and 7. Since low does not exceed high, the target substring is successfully found with one match. The suffix array reveals that this substring occurs at index 1 of the original string S. However, an evident drawback of the occ auxiliary structure is its high storage cost. Each character requires a column equal in length to the BWT. To mitigate this cost, occ can be stored at intervals of d columns, while the remaining columns are determined by calculating the frequency of specific characters in the corresponding segment of the BWT. Furthermore, since count and occ are often used together during exact pattern matching, [5] proposes merging these two data structures to reduce read overhead. This method compresses the content of every d entries into one, accompanied by the BWT result of the segment which is referred to in [5] as a “bucket”. For convenience, this paper will also refer to this structure as a bucket in subsequent segments.

2.1.3 Racetrack Memory

Racetrack Memory (RTM), also known as Domain-Wall Memory (DWM) [6], is a spintronic non-volatile memory technology constructed from ferromagnetic nanowires. These nanowires contain domains separated by domain walls, where each domain represents a single bit. The magnetic orientation of each domain, which is mutually exclusive, determines whether the bit represents a 0 or 1 based on whether the orientation is parallel or anti-parallel to a reference direction.

Unlike traditional volatile memory such as SSDs or emerging non-volatile memories like ReRAM [7], which allow direct random access to any data cell, RTM requires sequential access. In a single nanowire, multiple domains share one or more access ports, which are used for reading or writing operations. Accessing a specific domain requires two steps: (1) Shifting the target domain to align with the access port. (2) Applying an appropriate current (similar to Spin-Transfer Torque RAM, STT-RAM

[8]) to perform the read or write operation. The detailed structure and operational process of RTM are illustrated in Fig. 4. The dark-colored domains store data, while the light-colored domains serve as a buffer to prevent data loss during the shift operation. The black sections represent read-only or read/write ports. Read-only ports allow data retrieval via the RWL (Read Word Line) switch. Read/write ports utilize a technique known as shift write [9], where the direction of the current between BL (Bit Line) and BLB (Bit Line Bar) determines the magnetic orientation of the domain via the RWL switch. To align the port and the target domain during read operations, the SL (Shift Line) switch determines the shift direction. Voltage is applied from one end of the nanowire to the other to move the domain's data.

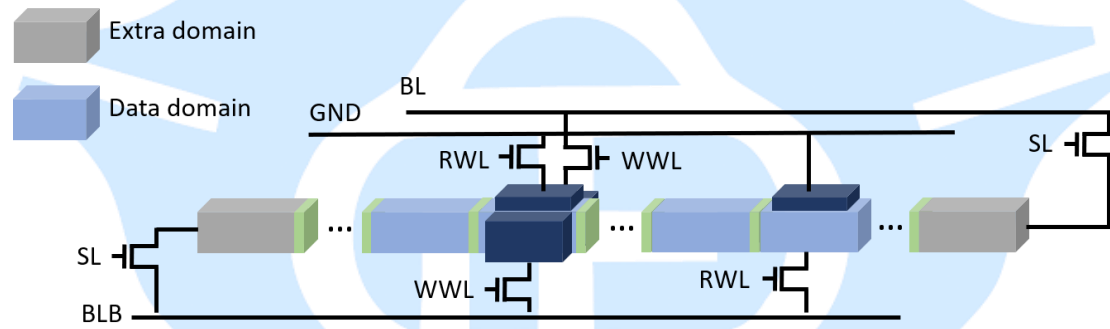


Fig. 4 : Anatomy of a RTM nanowire.

After understanding the basic architecture, operation, advantages and disadvantages of RTM, we now introduce two method of data placement in RTM as shown in Fig. 5a and Fig. 5b.

1. Word-Based Mapping : In word-based mapping, data is stored sequentially along the nanowire. Under this arrangement, the total number of operations during read or write access is influenced by two factors: the size of a word and the distance between the target word and the nearest access port. For example, assuming a word consists of 32 bits, to read word1, it must first be shifted to the closest access port. This requires shifting the entirety of word0. Afterward, the 32 bits of word1 are accessed sequentially

from the first bit to the last, resulting in a total of 64 shifts. This method clearly incurs a high number of shift operations. While adding more access ports can alleviate this issue, the additional ports would impose a significant spatial cost.

2. Bit-Interleaved Mapping : Bit-interleaved mapping distributes data across different tracks. When the target word is shifted to the access port, all tracks can perform read or write operations in parallel. For example, to read word1, only one shift is required to move word1 to the access port. Since the data is distributed across the tracks, all tracks can read simultaneously, eliminating the need for additional shifts. Minimizing the number of shifts is a core objective in designing RTM systems. Therefore, the architecture proposed in this paper adopts bit-interleaved mapping to arrange data, aiming to minimize the possibility of shift operations and enhance system efficiency.

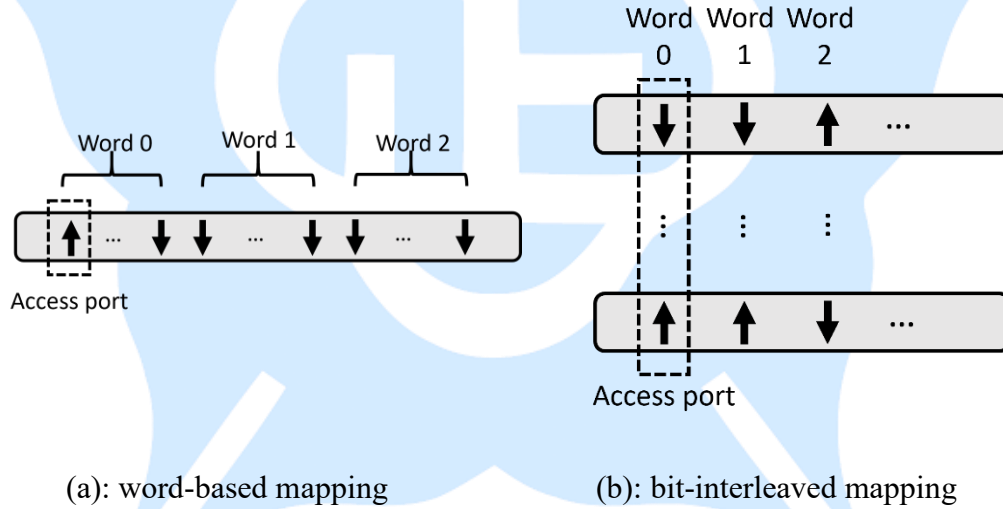


Fig. 5 : Data Mapping on RTM

2.1.4 RTM-base Main Memory Architecture

In prior designs using RTM as a standalone main memory, a hierarchical architecture was adopted to minimize modifications to the memory controller by maintaining the same I/O interface as DRAM. This approach has been employed in

other emerging memory technologies [12] and subsequently extended to RTM [13][14].

In this architecture, each bank is further divided into multiple subarrays, which are composed of identical-sized tiles, similar to traditional DRAM main memory. This design allows data within the same bank to be transferred efficiently. To facilitate RTM integration, each tile is composed of multiple RTM tracks grouped into a structure called a Domain Block Cluster (DBC). The DBC serves as the fundamental operational unit of RTM main memory. Within the same tile, all DBCs share a common set of sense amplifiers and write drivers. If data placement adopts the bit-interleaved mapping method, the number of tracks in each DBC is determined by the data word size. For instance, as shown in [14], a DBC configured for a word size of 32 bits would consist of 32 tracks. The length of each track can be adjusted according to the specific architectural requirements.

2.1.5 Transverse Read

Transverse Read (TR) [15] is a specialized operation in RTM, where a portion of a track is treated as a multi-level STT-RAM cell. By performing TR, the number of bits representing 1 between two access ports or within a single access port can be determined. However, TR cannot identify the precise positions of these 1 bits within the domains. This operation has certain constraints; as the number of domains increases, the minimum sense margin decreases, which imposes a limit on the number of domains involved in TR. The detailed operation is illustrated in Fig. 6.

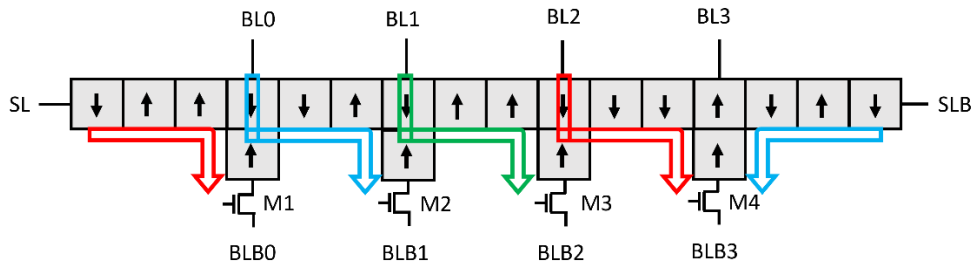


Fig. 6 : Transverse read example

In the scenario shown in Fig. 6, a complete track can perform searches using fragmented TR operations. This situation arises when the maximum number of domains that can be processed by TR is smaller than the length of the entire track. Each arrow in the figure represents the current path during a TR operation. For instance, consider the four domains between BL1 and BL2. At the start of TR, transistors M1, M2, and M4 are turned on, while M3 remains off. Consequently, the current flows from BL1, passes through the domains between BL1 and BL2, and exits the track via M3, completing one TR operation. Regions where TR can occur simultaneously are represented by arrows of the same style. For example, when the solid arrows are active, transistors M2, M3, and SLB are turned on, while M1 and M4 are turned off. During this phase, current emitted from SL and BL2 flows through adjacent, consecutive domains, exiting the track through M1 and M4 and reading out two and one 1 bits, respectively. The ability to perform parallel TR operations stems from the sufficient distance between BL0 and BL2, which generates enough impedance to minimize leakage current, thereby preventing interference with parallel TR operations. Previous research has leveraged the properties of TR to address misalignment errors during RTM shifts [16]. Additionally, some studies have utilized TR to perform bitwise operations within the maximum domain count of TR. Using a level-detecting sense amplifier, different voltage thresholds can be detected when 0, 1, ..., or n 1 bits appear on a track. For example, in a TR operation over four words: OR can be represented by any situation where the number of 1 bits exceeds threshold 1. AND can be represented by exceeding threshold 4. XOR can be represented by exceeding thresholds 1 and 3.

For a fixed TR length, the above logic can be used to execute sum and carry operations. Furthermore, TR can be employed to implement addition and multiplication operations [13][17].

2.2 Related Works

2.2.1 Processing In Memory

With the advancement of modern processors and the increasing demand for data required by various applications, significant challenges have emerged. For instance, in neural network models, the growing complexity of neural networks generates a substantial amount of intermediate computation results during processing, alongside rapidly expanding training datasets over time. These scenarios exemplify the growing bottleneck in the traditional Von Neumann architecture, where memory access speeds increasingly lag behind processor performance. To address this disparity, an effective architecture known as Processing In Memory (PIM) has been proposed. PIM integrates additional CMOS logic gates with conventional memory cells to form computational units, enabling data processing directly within memory. This approach eliminates the need to transfer data via the data bus to the processor, thereby significantly improving processing efficiency.

Previous research [18] has proposed dividing the memory architecture into RTM (Racetrack Memory) storage components and Skyrmion (SK) computational components. SK [19], a novel type of RTM, is partitioned in this manner due to the distinct advantages offered by each component. The RTM storage component provides benefits such as low leakage power, non-volatility, and high density, which make it ideal for storage applications. On the other hand, SK exhibits advantages over RTM when used as a basic logic element, requiring fewer CMOS peripheral circuits. The computational component includes SK-based adders and multipliers. Since addition operations require two XOR logic gates, and XOR gates cannot be directly implemented using SK, basic logic gates constructed from SK, such as NOR and NAND gates [20][21], are used to create XOR logic gates. This design enables the

construction of sum and carry operations, facilitating the implementation of full adders and multipliers. Another study [22] observed that neural network computations involve a significant amount of matrix multiplication. From the perspective of memory access, conventional volatile random-access memories such as SRAM and DRAM, or emerging non-volatile memories like ReRAM, allow efficient random access to any element. However, for RTM, random access is constrained by its inherent structure, requiring shifts to access the target element. To address this limitation, the study redesigned the data layout of matrices from the conventional one-dimensional array to a two-dimensional checkerboard layout optimized for SK-RM.

Some other studies leverage TR to enable processing-in-memory (PIM). For instance, [13] proposed a method utilizing RTM to perform multi-operand bulk-bitwise logic in a PIM architecture. Previous works using DRAM [25][26] for bitwise operations were limited to handling only two operands. While subsequent studies explored PIM implementations with other non-volatile memories, these were also restricted to two operands and faced various memory-specific challenges. For example, PCM and ReRAM suffer from low endurance and high write energy consumption, while STT-RAM struggles with insufficient memory density. This study utilizes the TR property to generate bitwise operation results representing various logic functions, such as basic logical operations, addition results, and carry outputs. To enable TR operations, the sense amplifier (SA) was modified, as shown in Fig 7(a). The output of the SA comprises seven bit-level values, where the output of $SA_i[j]$ equals 1 if the TR operation detects at least j "1"s, where $j \in 1 \dots 7$. These outputs from the SA serve as inputs to the PIM component, as illustrated in Fig. 7(b).

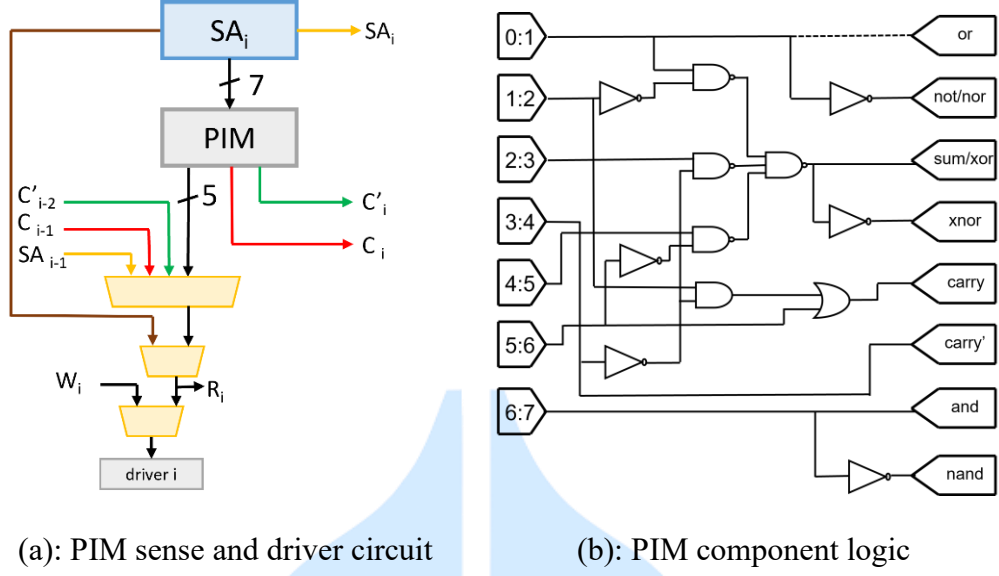


Fig. 7 : Overview of the PIM enabled DBC extensions

The proposed PIM component can perform bitwise operations on the same bit-line of multiple operands. If the resulting bit level exceeds one, the OR operation output will be 1, while the NOR output can be obtained by inverting the OR result. When the TR distance contains only one data bit with all other domains padded with 0, the outputs can represent as the NOT operation. AND/NAND logic results can be determined using the highest bit level, while XOR can be computed using relatively simple NAND/NAND-based implementations to determine odd bit level conditions. In addition to basic logical operations, the architecture also supports addition operations. The sum and XOR computations share the same output, while the carry is represented by bit level greater than or equal to two but not exceeding four, and exceeding six. Since the architecture supports up to seven operands, cross-bit carry generation is required. This carry is determined by bit levels exceeding four.

Above logical/addition outputs generated by the i -th, $(i - 1)$ -th and $(i - 2)$ -th PIM components are processed through a multiplexer to select the desired computation result. A subsequent 2-to-1 multiplexer allows switching between normal read outputs and PIM logic computation results. Both outputs are sent to the row buffer via the read port,

from which they are transmitted to the memory controller and eventually returned to the processor. An additional multiplexer enables the PIM result to be selected and written back to the memory cell through the write port. In summary, the discussed PIM architectures showcase effective strategies to reduce data movement and enhance computational efficiency by executing operations directly within memory.

2.2.2 Pattern Matching with PIM

Pattern matching is a fundamental technique widely applied in various fields, such as string processing and search within large-scale text data, as well as in the assembly and alignment stages of genome sequencing analysis. Pattern matching can be further categorized into two types: approximate pattern matching and exact pattern matching.

Approximate pattern matching algorithms typically consist of two main steps: filtering and verification. In the filtering step, numerous sub-patterns are generated and transferred from memory to the processor. During the verification phase, substantial edit distance (ED) computations are performed, requiring the construction of large dynamic programming matrices. These computations are often hindered by the limitations imposed by the memory wall problem, leading to degraded overall performance.

To address these challenges, [25] leverages ReRAM and adopts a PIM paradigm to mitigate the memory wall problem in both steps. Specifically, for the filtering phase, the study modifies the ReCAM architecture [26] to implement a q-gram filter. For the verification phase, a novel mapping approach tailored to ReRAM's architecture is proposed to efficiently compute edit distance.

Exact pattern matching plays a critical role in genome sequencing analysis, with algorithms such as super-maximal exact match (SMEM) widely adopted in practice. While BWA-MEM2 [27] is a prevalent industry-standard tool, its performance on short-

read processing suffers due to extensive random memory accesses, leading to significant latency and energy bottlenecks. To address this issue, CASA [28] proposes a SMEM seeding accelerator based on Content Addressable Memory (CAM). CASA integrates a filtering algorithm with an optimized data storage strategy to enhance computational efficiency and reduce energy consumption. Unlike conventional approaches that store the reference genome in DRAM and rely on index-table lookups, CASA stores the reference directly in CAM and employs a three-stage pre-seeding filter to eliminate invalid k-mers, thereby avoiding unnecessary SMEM computations. Moreover, CASA adopts a non-overlapping storage layout and a group-based activation scheme to further minimize CAM search energy. Experimental results show that CASA achieves up to $17\times$ throughput improvement over BWA-MEM2 using only 55 MB of on-chip memory.

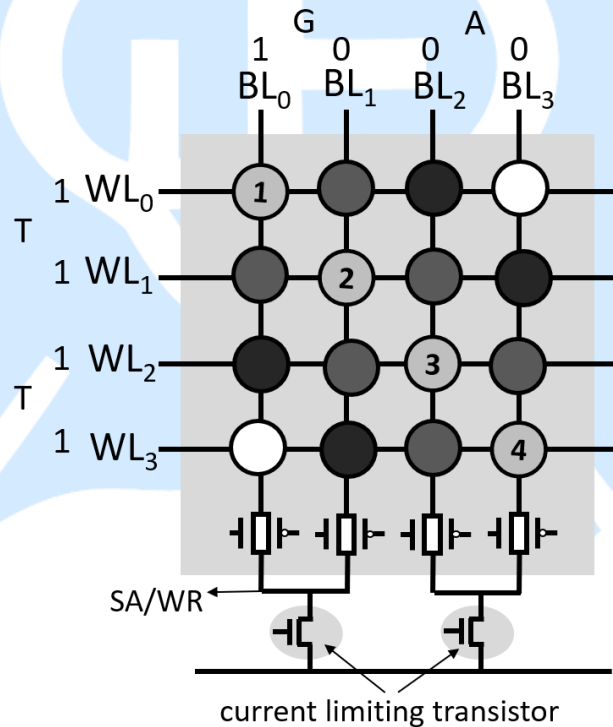


Fig. 8 : ReRAM-based Hamming Distance Unit

Another commonly used exact pattern matching method is the FM-index, which has gained popularity in recent years. However, its search process still suffers from the memory wall problem due to frequent memory accesses. To mitigate this issue, [5] proposes a PIM architecture based on ReRAM to accelerate FM-index matching. The enhanced FM-index search algorithm involves numerous counting operations, and prior studies have demonstrated how ReRAM can efficiently perform these operations by computing the Hamming distance between the query and target symbols. As illustrated in Fig. 8, if each symbol is encoded using two bits—with the bit line representing the query and the word line representing the target symbol—different voltages are applied based on the symbol. When the bit and word lines of a ReRAM cell share the same voltage, the cell maintains high resistance; otherwise, it exhibits low resistance. For example, in a configuration where four cells are used (Cells 1, 2, 3, and 4), the voltage settings may result in the resistance states being high, low, low, and low, respectively. Since the current transistor only allows one current path through a low-resistance cell, the total current reflects the number of mismatched bits. In this case, the resulting Hamming distance is 2. Compared to GPU-, ASIC-, or FPGA-based designs, this approach offers lower latency and reduced energy consumption.

This concept also inspires our research, showing that similar functionality can be achieved by exploiting the unique shift-based data access characteristics of RTM.

3. Motivation

Although the improved backward search algorithms have been proposed to mitigate the storing cost of auxiliary data structure, this approach necessitates frequent accesses to the partial BWT. Such memory access patterns may become a bottleneck as the memory access speed may not match the processor's processing speed, thereby degrading overall performance. Previous research [5] has utilized ReRAM with PIM techniques to accelerate this computation. Compared to other approaches leveraging GPUs, ASICs, or FPGAs, ReRAM demonstrated better efficiency. However, ReRAM inherently suffers from certain limitations in terms of cell size, write endurance, and write energy consumption. In contrast, Table 1 [10] presents a comparison between RTM, conventional volatile memories, and emerging non-volatile memories. In addition to inheriting the non-volatility advantage, RTM can achieve latency comparable to high-performance SRAM while exhibiting extremely low leakage power, high write endurance, and a smaller cell footprint. In particular, with respect to memory density, the cell size of RTM can be as small as $\leq 2 F^2$, whereas ReRAM requires 4–10 F^2 . This means that, in terms of the physical area required to store a single bit, RTM occupies only 20%–50% of that of ReRAM, demonstrating a clear advantage in storage density.

For example, to store 1 MB of data, RTM requires only about 16,777,216 F^2 of storage area, whereas ReRAM requires approximately 33,554,432–83,886,080 F^2 . In other words, under the same data capacity requirement, the storage cell area required by RTM is merely one-half to one-fifth that of ReRAM, which significantly reduces the chip area allocated to memory cells and frees up more die space for logic circuits or additional functionalities. This advantage is particularly valuable for applications where chip area is highly constrained—such as mobile devices, wearable electronics, edge

computing nodes, and highly integrated domain-specific accelerators—because it enables sufficient or even larger data storage within a smaller package.

Consequently, under the same chip area constraint, RTM can accommodate approximately 2–5x the storage capacity of ReRAM. This allows significantly larger datasets to be stored directly on-chip for data-intensive workloads such as genome sequencing analysis, graphics processing, and large-scale database queries, thereby reducing the frequency of off-chip memory accesses and minimizing both data transfer latency and energy consumption. This density advantage is especially critical for Processing-in-Memory (PIM) architectures, where a substantial portion of the performance bottleneck originates from data movement overhead. Storing more data in the same physical footprint can directly improve computational efficiency and throughput.

Moreover, RTM possesses unique operational capabilities. Prior research has utilized RTM for PIM operations, leveraging the TR characteristic to efficiently compare difference between two bits. This paper aims to exploit this property to compute the aforementioned symbol occurrence counts. Additionally, the overall architecture will be designed to address the constraints imposed by RTM’s characteristics.

Table 1 : Comparison of RTMs with other memory technologies

	SRAM	DRAM	STT-MRAM	ReRAM	PCM	RTM
Cell Size(F ²)	120-200	4-8	6-50	4-10	4-12	≤ 2
Write Endurance	≥ 10 ¹⁶	≥ 10 ¹⁶	4x10 ¹²	10 ¹¹	10 ⁹	≥ 10 ¹⁶
Read Time (ns)	1-100	30	3-15	10-20	5-20	3–250†
Write Time (ns)	1-100	30	3-15	20	> 30	3–250†
Write Energy	Low	Medium	High	High	High	Low
Read Energy	Low	Medium	Medium	Medium	Medium	Low
Leakage Power	High	Medium	Low	Low	Low	Low
Retention Period	Voltage dependent	64-512ms	Variable	Years	Years	Years

† include shift latency

4. RT-BPIM

4.1 Overview

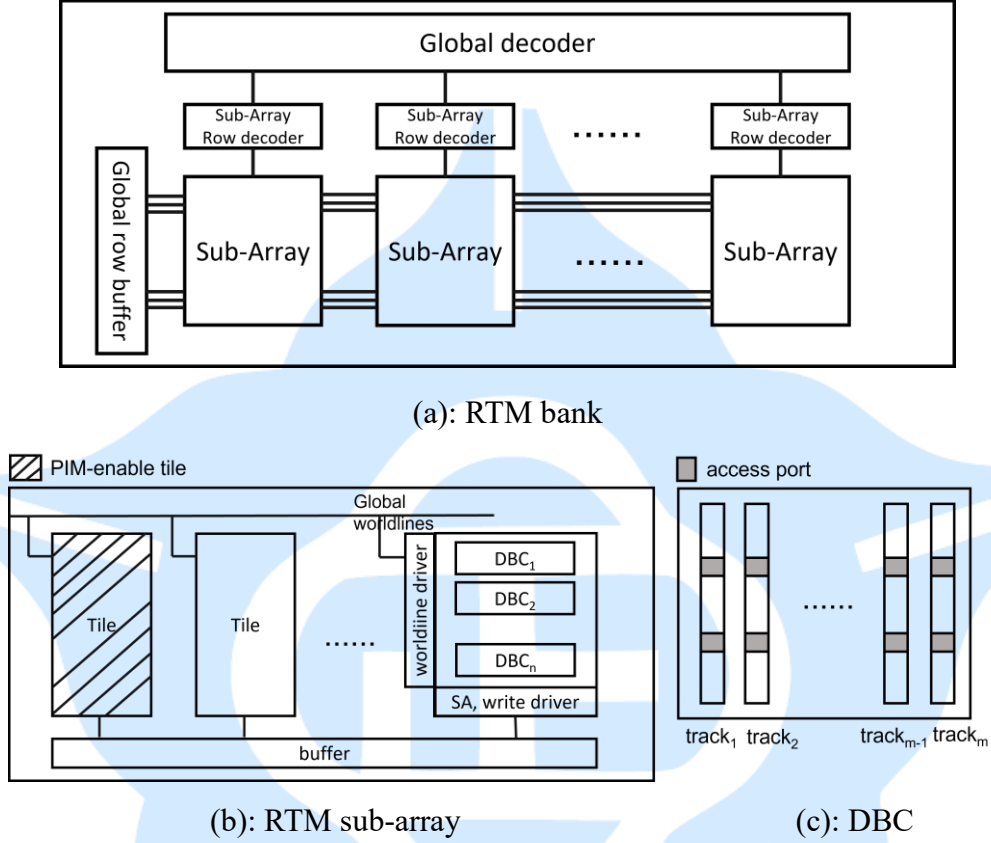


Fig. 9 : RTM Main Memory Architecture

To mitigate the challenges posed by intensive read operations in FM-index searches when leveraging RTM as the main memory, this paper proposes the RTM Bucket-Size Processing-In-Memory (RT-BPIM) architecture. As mentioned earlier, the RTM main memory is designed to minimize modifications to the memory controller, achieving compatibility by maintaining the same I/O interface as DRAM. Consequently, the proposed architecture adopts a hierarchical structure similar to that shown in Fig. 9. The overall architecture divides banks into multiple sub-arrays as illustrated in Fig. 9(a), which are further managed as tiles, as depicted in Fig. 9(b). Tiles in the same sub-array

share a common buffer. For facilitate integration, each tile consists of multiple DBCs which are composed of multiple tracks and act as the fundamental management unit of the main memory. Additionally, to reduce costs, all DBCs within a single tile share one set of Sense Amplifiers (SAs) and Write Drivers, thereby simplifying the hardware design and lowering implementation expenses.

When storing data, the BWT is arranged on the RTM in bit-interleaved manner. However, due to the inherent characteristics of RTM, shift operations incur significant overhead. To address this issue, Section 4.2 explores access patterns to the BWT during FM-index searches, while Section 4.3 introduces methods base on the bucket size to reduce the additional movement costs caused by shift operations. In FM-index searches, calculating low and high traditionally involves direct reads from the auxiliary structures count and occ. However, to reduce the storage overhead introduced by auxiliary structures, the improved backward search algorithm requires computing the symbol counts in the corresponding BWT segments during low and high calculations. To minimize the number of reads needed for these computations, this paper adopts a PIM approach, similar to prior research [5]. Modifications were made to the overall RTM main memory architecture to support this approach. Specifically, within each sub-array, one tile is designated as a PIM-enabled tile (as shown by the striped tile in Fig. 9b). This tile is responsible for performing the PIM operations described in Section 4.4. Some DBCs in the PIM-enabled tile handle PIM operations, while others continue to be used for regular data storage. Additionally, as mentioned earlier, DBCs within the same tile share the same set of SAs and Write Drivers. The proposed PIM architecture ensures that the PIM-enabled tile can execute PIM operations while maintaining normal read functionality. A detailed explanation of this architecture is provided in Section 4.4.

4.2 Query of FM-index and Counting algorithm

In the FM-index search, auxiliary structures optimized for storage, as shown in Fig. 10, significantly reduce the storage space required compared to the original structure.

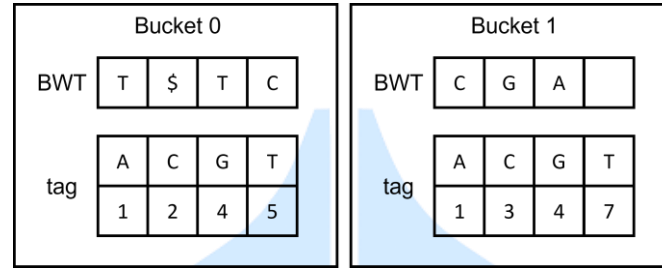


Fig. 10 : Storage Optimized FM-index

Taking Fig. 6 as an example, the pre-optimized occ structure requires eight fields. After optimization, only two fields are needed by storing values every four fields instead. However, this modification necessitates adjustments to the backward search process.

In the original algorithm, computing low and high only requires direct access to occ and count (as shown in Algorithm 1). With the optimized auxiliary structure, if the computed field is not a tag, the number of symbols in the corresponding bucket's BWT must be counted and added to the tag to determine the next low/high position. This process is detailed in Algorithms 2 and 3. The Low-High Mapping (LHM) function one line 6 and 7 takes the required symbol and the previous low/high position, identifies the corresponding BWT segment, calculates the symbol's occurrence, and adds it to the associated tag to compute the new low/high position.

However, this approach introduces a significant overhead in accessing the BWT during computation. The placement of the BWT thus has a considerable impact on access efficiency. The most straightforward approach resembles the hot/cold data organization, where frequently accessed BWT segments are grouped to improve

efficiency. Unfortunately, observations during the backward search reveal that FM-index access patterns are almost entirely random, making data placement optimization ineffective. Additionally, maintaining the correctness of the BWT further complicates data mapping. Given this, arranging data in a bit-interleaved manner provides the optimal approach. Data should be placed sequentially along the first domain of each track and then across the remaining domains. For traditional RTM main memory, where each DBC's capacity equals the size of a word, no significant issues arise when the word size exceeds the bucket size. However, when the word size is smaller than the bucket size, additional shift operations are required to access the entire bucket, reducing overall efficiency. The next section 4.3 will introduce methods to address this issue and further discuss the impact of these improvements on the backward search algorithm.

Algorithm 2 Backward Search (BWT, Q)

```

1: Input: BWT (Burrows-Wheeler Transform), Q (Query string)
2: Output: Matching interval or termination if no match is found
3: low  $\leftarrow$  0
4: high  $\leftarrow$  len(occ) - 1
5: for i  $\leftarrow$  len(Q) - 1 to 0 do
6:   low  $\leftarrow$  LHM(Q[i], low, BWT)
7:   high  $\leftarrow$  LHM(Q[i], high, BWT)
8:   if high < low then
9:     return
10:  end if
11: end for

```

Algorithm 3 LHM (s, x, BWT)

```

1: Input: s (Symbol), x (Previous position), BWT
2: Output: Next mapping position
3: count  $\leftarrow$  0
4: for i  $\leftarrow$  0 to (x mod d) - 1 do
5:   if BWT[(x / d * d) + i] == s then
6:     count++
7:   end if
8: end for
9: return tag[x / d][s] + count

```

4.3 Shift Optimization Design

Due to the inherent characteristics of RTM, data access requires moving the data to the access port before it can be accessed. Unlike other storage devices that can perform random access with ease, RTM requires additional shifts, resulting in access latency and energy consumption being heavily influenced by these extra shifts. In the worst-case scenario, the access latency of RTM can be up to 25.6 times higher than that of an SRAM of equivalent capacity [11]. Therefore, reducing the number of required shifts is a critical factor in improving search efficiency.

As discussed in the previous section, the data arrangement method plays an essential role. In typical RTM Main Memory, the DBC serves as the basic unit of access, with each DBC containing tracks whose number usually corresponds to the size of a word. Given that the data is mapping in bit-interleaved and sequentially distributed along the domains of each track, certain challenges arise. Assume the bucket size is 256 and that symbols commonly used in genome sequencing, such as A, C, G, and T, are each represented by 2 bits. Further assume that the word size is 32 bits. Accessing the entire bucket would require, in addition to the shifts needed to reach the bucket's starting position, 15 additional shifts to read the entire bucket. To mitigate the shift cost, a straightforward approach would be to increase the number of tracks within each DBC. This reduces the likelihood of shifts occurring by allowing more data to be accessed per shift. However, as observed in Section 4.2, the FM-index's access pattern for BWT segments during the computation of low/high positions is almost entirely random access. Even if the number of tracks were to encompass multiple buckets, the contiguous nature of BWT segments between buckets would still necessitate additional shifts. Furthermore, excessive tracks might bring unnecessary BWT segments to the access port, leading to increased energy consumption. Hence, an unrestricted increase

in track length is clearly not a viable solution.

A better approach involves configuring the number of tracks according to the bucket size. In this manner, during computation, only the shifts required to move the relevant bucket to the access port are needed, without incurring additional shifts. Moreover, as only one bucket is shifted to the access port at a time, the issue of unnecessary buckets being shifted is avoided. However, while accessing a BWT segments for symbol counting, it is not always necessary to process all symbols in the bucket. In the best-case scenario, if the position happens to store a tag, symbol counting for the BWT segments can be skipped entirely. In the worst-case scenario, using a bucket size of 256 as an example, up to 255 symbols may need to be counted. After some Observations, similar to the randomness in low/high positions, the number of symbols requiring counting exhibit an almost random pattern. The occurrence of the best-case scenario, where no BWT segments access is needed, is not very frequent but not uncommon either. Although using a bucket size as the track length might still bring unnecessary symbols into the access port, overhead is significantly smaller compared to the cost of unrestrictedly increasing the number of tracks. Furthermore, compared to the number of track based on word size, configuring the number of tracks according to bucket size effectively reduces the number of shifts required.

4.4 RTM-base PIM Counting

After the shift optimization for BWT segments searches, the next step involves the computation of the symbol counts. In conventional RTM main memory, due to the limitations of the CPU data bus, the BWT segments used for symbol counting can only be read in segments corresponding to the data bus length. Assuming a data bus length of one word (32 bits) and a bucket size of 256, with each symbol represented by 2 bits,

a worst-case scenario requiring the entire bucket to be read would necessitate 16 read operations. Such excessive reads result in the issues discussed in Chapter 2, where the overall read speed cannot keep up with the processor's processing speed, thereby reducing overall efficiency.

Prior research [5] addressed this issue by performing symbol counting directly within ReRAM, effectively mitigating the aforementioned problem. However, due to the intrinsic properties of RTM, operations using processing-in-memory (PIM) have significant potential. Thus, this section introduces a method for performing symbol counting directly within RTM. Section 4.4.1 first explains how to move the BWT section to the PIM DBC and describes the details of the PIM DBC itself. Subsequently, Section 4.4.2 provides a detailed explanation of how transverse reads can facilitate the process of symbol counting.

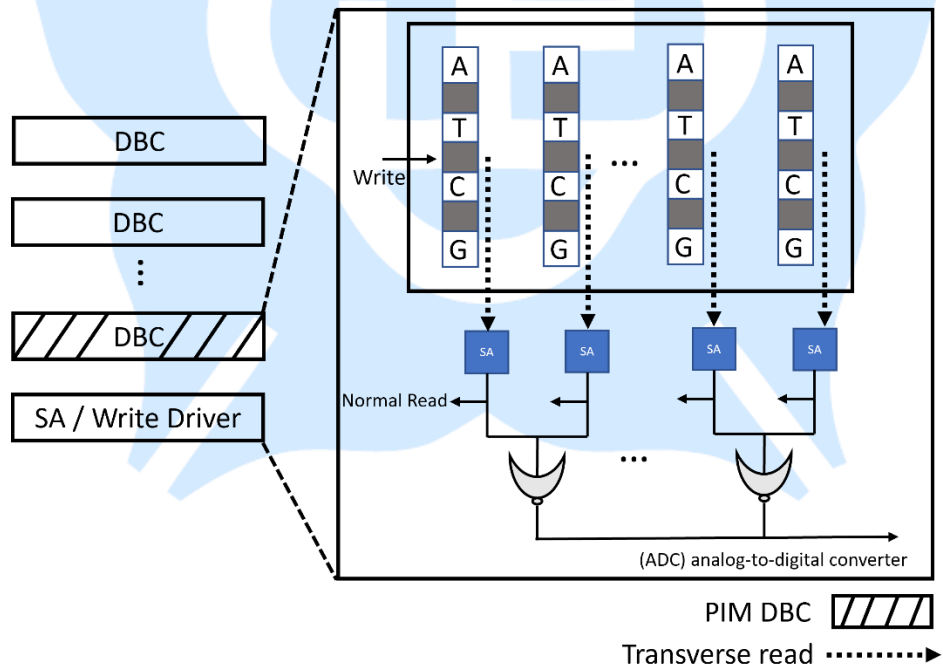


Fig. 11 : PIM DBC and TR Counting

4.4.1 Details of PIM DBC

After shifting the target bucket to the access port, the next step is to calculate the symbol count for the entire bucket. This requires moving the BWT segments within the bucket from the storage DBCs to the computation DBCs designated for symbol counting. Each tile contains one DBC dedicated to symbol counting (as shown in Fig. 11, marked with stripes), while the remaining DBCs serve as standard storage for BWT. During the computation process, the target bucket is first read from the storage DBCs into a buffer and then written into the PIM DBC. The structure of the PIM DBC is depicted in Fig. 11. The symbols to be counted are pre-written into specific positions within the DBC corresponding to the symbols being calculated. This preloading reduces the time required for symbol writing, as the only remaining write operation involves the target bucket. Furthermore, due to the bit-interleaved arrangement, the entire bucket can be written synchronously, making the write latency equivalent to a single write operation.

Reducing the occurrence of shifts is a core concept of the RTM architecture. As the process only involves writing the bucket to designated symbol positions, no additional shifts occur during the operation. While preloading symbols consumes additional space, it significantly reduces write and shift operations, thereby improving overall search efficiency. Since TR can operate between two access ports, it further enhances the symbol counting process. For instance, as shown in Fig. 11, if the symbol to be counted is "C," the bucket is written to the second access port from the top. This enables TR to be performed between the second and third ports to complete the operation. TR can also be initiated from either end of the access port or track. For example, if the symbol to be counted is "A," the bucket is written into the first access port. The TR operation is then performed through the domain storing symbol "A" and

finalized at the track's endpoint. Furthermore, as only a subset of symbols within the entire bucket may be required, specific tracks can be selectively activated to perform operations, thereby preventing calculation errors. The following subsection will describe how the results of TR are utilized to complete the symbol counting process.

4.4.2 In Memory symbol counting

Once the TR results are obtained, the next step is to perform symbol counting based on these results. Symbol counting determines how many symbols within a bucket match the target symbol. The TR results can be categorized into three cases: 0, 1, or 2 occurrences of "". Here, a "1" indicates a bit mismatch between two bits, which can either be (1, 0) or (0, 1). Conversely, when the bits are identical, the TR result will be 0 or 2, signifying that both bits are either (0, 0) or (1, 1). These outputs serve as the basis for identifying symbol matches. For example, in the context of genomic sequencing, symbols such as A, C, G, and T can be encoded using two bits each. During the TR operation, if the result is "1," the output of the Sense Amplifier (SA) remains 1, signifying a mismatch. However, if the TR result is "0" or "2," the SA output transitions to 0, indicating no mismatch. The SA output is subsequently directed to a NOR gate for further processing. When a symbol has at least one mismatched bit, the two symbols are deemed different, and the NOR gate outputs a 0. Conversely, if all bits of the two symbols match perfectly, the NOR gate's output becomes 1. This logic ensures that only symbols with all bits matching the target symbol contribute to the final count.

Parallel Processing for Efficiency

Due to the bit-interleaved mapping utilized in this architecture, the comparison results for all symbols within a bucket can be computed simultaneously. This parallel computation significantly enhances the efficiency of the counting process. Once the comparison results for all symbols are determined, the outputs are processed through

an Analog-to-Digital Converter (ADC) [29] to produce the final symbol count, as described in previous studies [5]. The ADC enables accurate conversion of the parallel analog outputs into a digital format suitable for further processing or storage.

Resource Optimization and DBC Modifications :

The entire symbol counting process involves four primary steps:

1. Bucket Symbol Read: Symbols are read from the storage DBC into a buffer.
2. Write to PIM DBC: The symbols are written into the PIM DBC.
3. TR Operation: Transverse read is performed to identify bit mismatches.
4. ADC Conversion: Outputs are converted to digital format for counting.

Notably, this efficient process minimizes latency by leveraging parallelism at every step.

To maintain the general storage functionality of DBCs, all DBCs within a tile share a common SA/Write Driver set. Within a PIM enable tile, additional components, including modified SAs, comparison NOR gates, and an ADC, are integrated, as depicted in the left half of Fig. 11. These modifications enable the PIM DBC to perform symbol counting while preserving compatibility with traditional storage DBC operations.

Compatibility with Traditional Storage DBCs :

Despite the additional components, the modified SAs function identically to standard SAs for TR results of 0 or 1 "1." This ensures that even traditional storage DBCs within the PIM tile can perform standard read operations. This dual functionality is crucial for maintaining the overall efficiency and flexibility of the architecture, enabling both symbol counting and general storage tasks within the same hardware framework. By combining these optimizations and leveraging the inherent parallelism of bit-interleaved mapping, this architecture achieves high efficiency and scalability for symbol counting operations, which is critical for applications such as genomic sequencing and large-scale data processing.

5. Experiment and Evaluation

5.1 Experiment Setup

5.1.1 Simulation and Configuration

The proposed RT-BPIM architecture primarily addresses the inefficiencies in the symbol counting process of FM-index, which is a crucial and resource-intensive component. To evaluate the performance of RT-BPIM, the experiments are designed to focus on the costs associated with individual operations, such as Shift, Read, Write, and TR. Since large-scale physical RTM systems are currently unavailable for practical experimentation, a custom-developed simulator is employed. This simulator is capable of emulating RTM behaviors, including memory access and PIM operations, and operates as a trace-based simulator.

The simulation assumes that all data has been preloaded into the RTM main memory. Input traces are processed through an address translator, converting logical addresses into physical locations within the RTM. The simulator then identifies the specific portions of RTM to be accessed and performs the required operations. The simulator fully models the bank, subarray, tile, and DBC structures of the RTM. The DBC module includes access port positional information, which is updated dynamically based on ongoing operations. The number of operations, including Shift, Read, Write and TR, is recorded, and the access port's location is updated accordingly during the simulation.

The experiments are conducted under three configurations, all utilizing RTM as the main memory. To focus solely on the efficiency of symbol counting and to keep simplicity, the memory system is simplified to a single bank, with each subarray containing four tiles and each tile comprising eight DBCs. The configurations are as follows:

1. Baseline CPU: This baseline configuration employs a CPU to perform symbol counting. It assumes a CPU data bus width of 32 bits, and the number of tracks on each DBC is set to match the data bus width. This setup represents the conventional system for baseline performance comparison.

2. Bucket Size CPU: This configuration evaluates the effectiveness of the optimized shift architecture. The number of tracks per DBC is dynamically adjusted based on the bucket size, while symbol counting remains executed by the CPU. This setup assesses the impact of reduced shift operations on overall efficiency.

3. Bucket Size PIM: This configuration builds upon Configuration 2 by maintaining the track settings based on bucket size. However, the symbol counting is delegated to the PIM-enabled DBCs, leveraging the advantages of Processing-In-Memory.

The parameters used in the simulation are based on data provided by previous studies, as summarized in Table 2. Since RT-BPIM is primarily designed to address the issue of excessive memory reads, the experiment focuses on counting the number of Shift, Read, Write, and TR operations during the search process. The latency incurred in Configuration 1 due to data transfer to the CPU for computation is excluded from consideration, as the CPU's processing speed is significantly faster compared to memory access latency and can be considered negligible. This exclusion also allows for a clearer evaluation of the improvements offered by RT-BPIM.

Table 2 : Latency and Energy consumption

Operation	Shift	Read	Write	TR
Latency (ns)	2.14	1.61	4.84	5
Energy (nJ)	0.019	0.08	0.108	0.175

5.1.2 Dataset and Trace Generate

For the experimental dataset, we adopted the human genome GRCh38 dataset, consistent with prior research [5], to conduct FM-index searches. This dataset comprises multiple human genomic sequences, with sequence lengths ranging from several million to several hundred million bases. The GRCh38 dataset is chosen for its comprehensiveness and relevance to practical genomic applications.

The search process employs a short-read approach, where a typical short read consists of a few hundred base pairs (bp). In this study, the length of short reads is set to 350 bp, and they are generated by randomly selecting segments from the aforementioned sequences. This ensures that the input reflects diverse genomic regions for a thorough evaluation of FM-index performance.

The trace generation process involves the following steps. First, the BWT of the target genomic sequence is generated. The resulting BWT sequence is treated as a continuous memory block. Next, short reads generated as described above are used as input for FM-index searches. Upon locating the starting positions of the corresponding bucket in the BWT, their memory addresses are recorded in a trace file. This process is repeated until all short reads have been processed. The generated trace file serves as the input for the trace-based simulator mentioned in the previous section.

5.2 Experiment Evaluation

5.2.1 Latency Evaluation for Symbol Counting Operations

Since the primary overhead in FM-index searches arises from memory access during symbol counting, this section analyzes the latency of symbol counting after applying shift optimization and in-memory counting operations. The analysis focuses on various RTM access operations, including Shift, Read, Write, and TR. Furthermore, as outlined in the previous section, the latency incurred by the CPU during the counting

process is excluded from the analysis in the CPU counting configuration. This exclusion allows for a clearer comparison of memory access efficiency across different configurations.

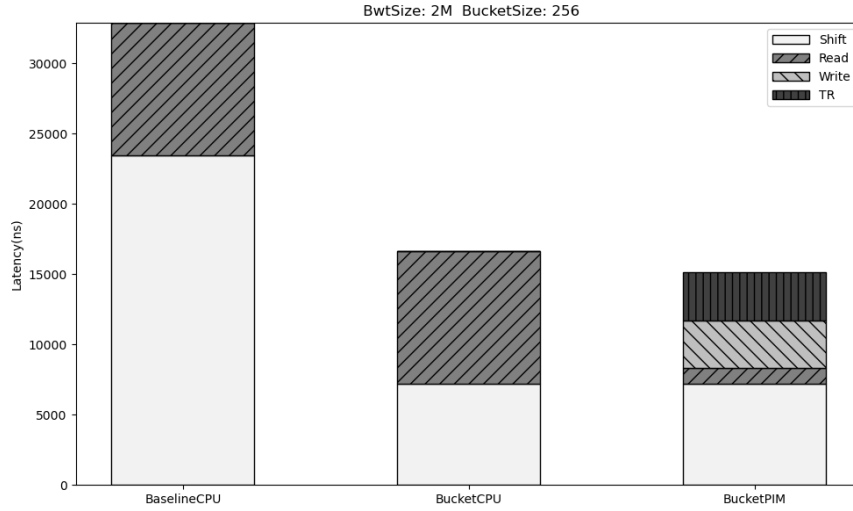


Fig. 12 : Latency of symbol counting with bucket size 256

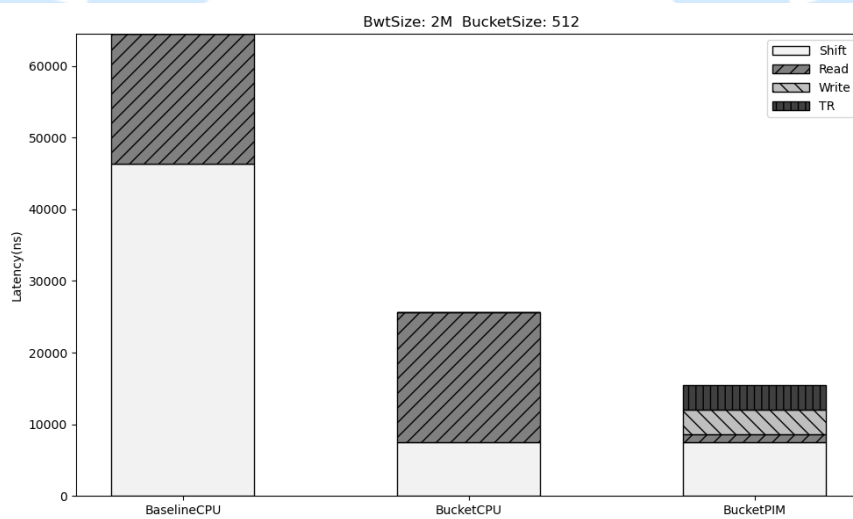


Fig. 13 : Latency of symbol counting with bucket size 512

Starting with the shift operations, it is obvious that the baseline configuration requires significantly more shifts compared to the configuration where the number of tracks is based on bucket size. This is because the baseline configuration allocates the number of tracks according to the size of a word. During the search process, the low/high computation can encounter three scenarios. The first scenario, which is the

most favorable, occurs when the accumulated symbol counting is directly stored in the tag, eliminating the need to read the BWT for symbol counting. However, if the value is not in the tag, symbol counting is required, leading to the other two scenarios.

In the second scenario, the size of the BWT segment required for symbol counting is less than or equal to one word. In this case, the shifts needed is to move the BWT segment to the access port. In the third scenario, the size of the required BWT segment exceeds one word. This situation necessitates not only the shifts to move the BWT segment to the access port but also additional shifts to read the portion of the segment that exceeds the word size. In the worst-case scenario, it may be necessary to read the entire BWT segment in the bucket. In contrast, the configuration where the number of tracks based on bucket size avoids the third scenario entirely. Regardless of the size of the required BWT segment, only the shifts to move the target bucket to the access port is needed. Since the number of tracks matches the bucket size, additional shifts are unnecessary. Figure 11 demonstrates that this configuration reduces the shift latency by approximately 67%.

Next, considering the symbol counting process, the first step involves reading the required BWT segment. In the baseline configuration, the length of the BWT segment exceeds the number of tracks, necessitating multiple reads following the shifts described earlier. For the second configuration, the symbol counting is performed by the CPU and is constrained by the size of the data bus (assumed to be 32 bits in the experiments). When the BWT segment exceeds the size of the data bus, the process involves sequentially activating tracks equal to the data bus size to perform reads, followed by activating subsequent tracks until all required symbols are read. As a result, the total number of reads is primarily influenced by the data bus size, similar to the baseline configuration. The third configuration, involving PIM-enabled DBCs for symbol counting, is not limited by the data bus size. Once shifting target bucket under

the access port, the entire required BWT segment can be read directly and written into the PIM-enabled DBCs, followed by the TR operation. Due to the bit-interleaved mapping, each symbol in the BWT segment can be processed in parallel. Although the third configuration can process the entire BWT segment at once, the latency of the write and TR operations is higher than that of the read operations. Consequently, when the latency of the reads required for the BWT segment is lower than that of the write and TR operations, the PIM configuration fails to demonstrate an advantage. Conversely, if the length of the BWT segment significantly exceeds the word size, the latency caused by multiple reads increases substantially, giving the PIM configuration a performance advantage.

The length of the required BWT segment within the bucket is, therefore, a critical factor influencing the efficiency of the counting process. As mentioned in Section 4.2, the FM-index access pattern during the low/high computation in backward search are almost entirely random access. This implies that the length of the required BWT segment for each low/high computation is also nearly random. If the expected length of the BWT segment results in read latency exceeding the combined latency of the write and TR operations, the PIM configuration can achieve superior performance. From Fig. 12, it is observed that when the bucket size is 256, the overall latency of the third configuration improves by approximately 8% compared to the second configuration. This indicates that under the condition of a bucket size of 256, the expected length of the BWT segment causes the read latency of the second configuration to surpass the overall latency of the third configuration, enabling the third configuration to achieve better performance.

When the bucket size increases from 256 to 512, the improvement margin also increases. The latency of the third configuration improves by approximately 75% and 39% compared to the first and second configurations, respectively. In the first

configuration, the increase in bucket size leads to a larger expected length of the BWT segment, resulting in a higher number of shifts required to read the entire BWT segment. However, in the bucket-size-based track configuration, the number of shifts remains constant regardless of the bucket size, thereby increasing the improvement margin. For the second configuration, the increased BWT segment length also results in a higher number of reads, whereas the third configuration maintains a constant number of operations regardless of the bucket size. Consequently, the improvement margin of the third configuration increases from 8% to 39%. The results from Fig. 13 indicate that as the bucket size increases, the efficiency of the third configuration also improves. However, this does not imply that the bucket size can be increased indefinitely. Energy consumption must also be considered, which will be discussed in the next section.

5.2.2 Energy Consumption Analysis

This section analyzes the energy consumption of different configurations, with the calculation based on the total energy consumed by all operations performed on tracks involved in symbol counting. The experimental results are shown in Fig. 14, 15.

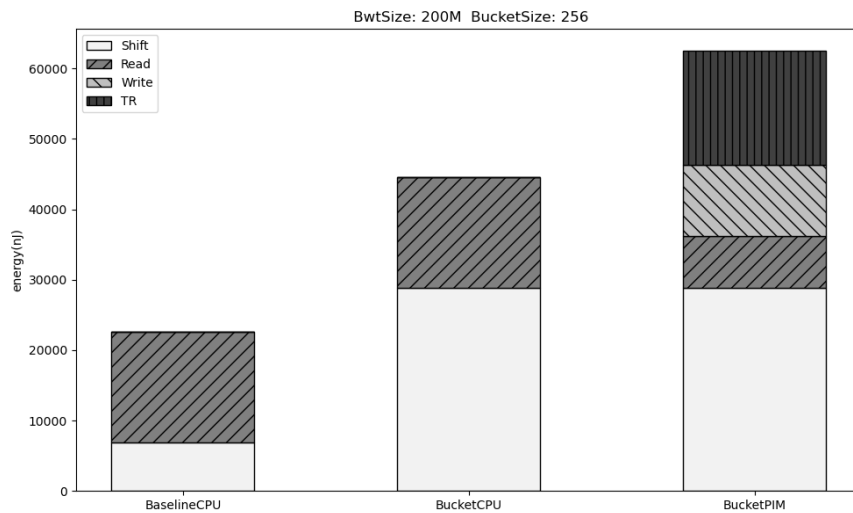


Fig. 14 : Energy consumption of symbol counting with bucket size 256

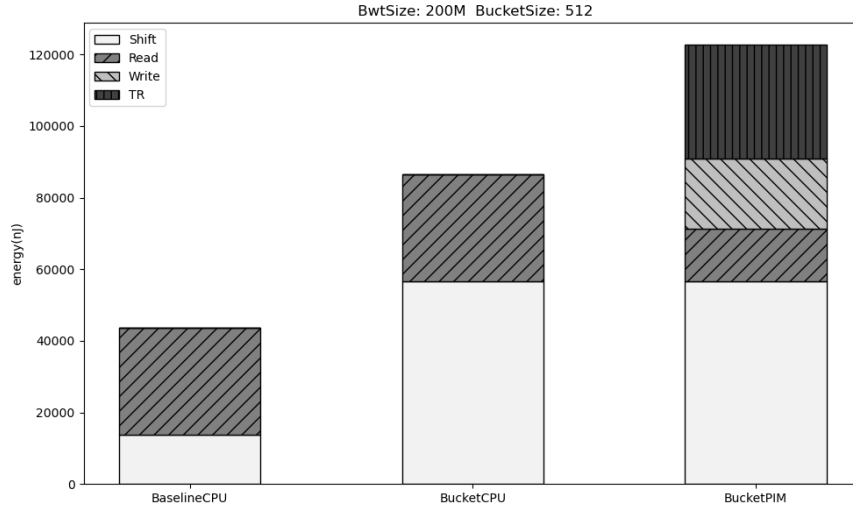


Fig. 15 : Energy consumption of symbol counting with bucket size 512

Fig. 14 presents the results for a bucket size of 256. Starting with the shift operations, the energy consumed by the bucket-size-based track configuration is approximately 4.17x higher than that of the word-size-based track configuration. This discrepancy arises because, in the bucket-size track configuration, the entire BWT segment is shifted to the access port regardless of how many sections within the bucket are utilized. Conversely, in the word-size track configuration, shifting ceases once the required BWT segments are read, leading to lower energy consumption.

For the symbol counting operations, the CPU-based configuration reads one word at a time, potentially loading unnecessary BWT segments into memory, thereby increasing energy consumption. In contrast, the PIM-based configuration activates specific tracks corresponding to the required BWT segments, reducing energy consumption during read operations. However, the write and TR operations inherently consume more energy, resulting in higher overall energy consumption for the PIM configuration. As shown in Fig. 14, the overall energy consumption of the third configuration is approximately 2.77x and 1.4x higher than the first and second configurations, respectively.

As illustrated in Fig. 15, increasing the bucket size to 512 further reduces latency

but nearly doubles the overall energy consumption. Therefore, in designing architectures, a trade-off between latency and energy efficiency must be carefully considered to meet specific application requirements.

5.2.3 PIM DBC Overhead Analysis with Varying Bucket Sizes

The primary additional overhead introduced by the proposed RT-BPIM architecture lies in the PIM DBC. As demonstrated in the experimental results from previous sections, indiscriminately increasing the bucket size is not an optimal approach. Therefore, this section focuses solely on the overhead introduced by the PIM DBC across varying bucket sizes. The experimental results are presented in Table 3.

Table 3 : Overhead Increase (%) of PIM DBC vs. Non-PIM Across Bucket Sizes

Bucket size	128	256	512	1024
Overhead	3.3%	3.3%	3.3%	3.3%

From the results, it can be observed that, when normalized against the baseline non-PIM architecture, the overhead increase is approximately 3.3%. This is because the PIM DBC within the same sub-array can be shared by multiple storage DBCs, thereby preventing significant overhead. Furthermore, the overhead remains consistent across different bucket sizes. While it might seem intuitive that increasing the bucket size would proportionally increase the size of the PIM DBC, thereby raising the overhead but the capacity of the storage DBCs also increases correspondingly. As a result, the overall overhead remains unchanged.

6. Conclusion

To address the excessive memory accesses that lead to the “memory wall problem” in exact pattern matching algorithms, this study proposes an architecture called RTM Bucket-Size Processing-In-Memory (RT-BPIM) with RTM as the main memory system to accelerate the matching process. Due to the inherent characteristics of RTM, accessing arbitrary blocks requires additional shift operations. By integrating an optimized FM-index structure, the number of tracks in each DBC is configured according to the bucket size, allowing all data within a bucket to be shifted to the desired position in a single operation. This approach significantly reduces the number of shifts required, although it may result in unnecessary data being moved, causing slight additional energy consumption. Nevertheless, it achieves a 67% reduction in shift latency compared to the baseline.

Additionally, as the matching process involves symbol counting, a PIM-based approach for symbol counting is introduced. RTM features a unique operation called TR, which allows symbol counting to be performed without transferring symbols to the CPU for computation. Instead, BWT segments are written to the PIM DBC, with the target symbols written to their corresponding positions. TR operations are then performed, and a minimal number of logic gates are used to determine the symbol count. While PIM operations may incur higher latency for shorter BWT segments, the nature of FM-index searches means that such cases are rare. Consequently, PIM operations generally exhibit better performance.

Experimental results demonstrate that RT-BPIM achieves a 75% reduction in overall search latency compared to the baseline, highlighting its effectiveness in addressing the memory wall problem and enhancing search efficiency.

REFERENCE

- [1] M. Schindler, “A fast block-sorting algorithm for lossless data compression,” in *Proc. Data Compression Conf. (DCC)*, Mar. 1997, pp. 469.
- [2] U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” *SIAM J. Comput.*, vol. 22, no. 5, pp. 935–948, 1993.
- [3] G. Nong, S. Zhang, and W. H. Chan, “Linear suffix array construction by almost pure induced-sorting,” in *Proc. Data Compression Conf.*, Mar. 2009, pp. 193–202.
- [4] P. Ferragina and G. Manzini, “An experimental study of an opportunistic index,” in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2001, pp. 269–278.
- [5] F. Zokaee, M. Zhang, and L. Jiang, “Finder: Accelerating FM-index-based exact pattern matching in genomic sequences through ReRAM technology,” in *Proc. 28th Int. Conf. Parallel Architectures and Compilation Techniques (PACT)*, 2019, pp. 29–42.
- [6] S. S. Parkin, M. Hayashi, and L. Thomas, “Magnetic domain-wall racetrack memory,” *Science*, vol. 320, no. 5873, pp. 190–194, 2008.
- [7] H. Akinaga and H. Shima, “Resistive random access memory (ReRAM) based on metal oxides,” *Proc. IEEE*, vol. 98, no. 12, pp. 2237–2251, 2010.
- [8] K. L. Wang, J. G. Alzate, and P. K. Amiri, “Low-power non-volatile spintronic memory: STT-RAM and beyond,” *J. Phys. D: Appl. Phys.*, vol. 46, no. 7, p. 074003, 2013.
- [9] R. Venkatesan, M. Sharad, K. Roy, and A. Raghunathan, “DWM-Tapestri—An energy efficient all-spin cache using domain wall shift based writes,” in *Proc. 2013 Design, Automation & Test in Europe Conf. Exhib. (DATE)*, Mar. 2013, pp. 1825–1830.
- [10] R. Bläsing, A. A. Khan, P. C. Filippou, C. Garg, F. Hameed, J. Castrillon, and S.

- S. Parkin, “Magnetic racetrack memory: From physics to the cusp of applications within a decade,” *Proc. IEEE*, vol. 108, no. 8, pp. 1303–1321, Aug. 2020.
- [11] R. Venkatesan, S. G. Ramasubramanian, S. Venkataramani, K. Roy, and A. Raghunathan, “Stag: Spintronic-tape architecture for GPGPU cache hierarchies,” *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 3, pp. 253–264, Jun. 2014.
- [12] J. Yue and Y. Zhu, “Exploiting subarrays inside a bank to improve phase change memory performance,” in *Proc. 2013 Design, Automation & Test in Europe Conf. Exhib. (DATE)*, Mar. 2013, pp. 386–391.
- [13] S. Ollivier, S. Longofono, P. Dutta, J. Hu, S. Bhanja, and A. K. Jones, “CORUSCANT: Fast efficient processing-in-racetrack memories,” in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2022, pp. 784–798.
- [14] A. A. Khan, F. Hameed, R. Bläsing, S. Parkin, and J. Castrillon, “RTSim: A cycle-accurate simulator for racetrack memories,” *IEEE Comput. Archit. Lett.*, vol. 18, no. 1, pp. 43–46, 2019.
- [15] K. Roxy, S. Ollivier, A. Hoque, S. Longofono, A. K. Jones, and S. Bhanja, “A novel transverse read technique for domain-wall ‘racetrack’ memories,” *IEEE Trans. Nanotechnol.*, vol. 19, pp. 648–652, 2020.
- [16] S. Ollivier, D. Kline, R. Kawsher, R. Melhem, S. Banja, and A. K. Jones, “Leveraging transverse reads to correct alignment faults in domain wall memories,” in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Jun. 2019, pp. 375–387.
- [17] A. A. Khan, S. Ollivier, S. Longofono, G. Hempel, J. Castrillon, and A. K. Jones, “Brain-inspired cognition in next-generation racetrack memories,” *ACM Trans. Embed. Comput. Syst.*, vol. 21, no. 6, pp. 1–28, 2022.
- [18] B. Liu, S. Gu, M. Chen, W. Kang, J. Hu, Q. Zhuge, and E. H. M. Sha, “An efficient racetrack memory-based processing-in-memory architecture for convolutional

- neural networks,” in *Proc. 2017 IEEE Int. Symp. Parallel and Distributed Processing with Applications and IEEE Int. Conf. Ubiquitous Computing and Communications (ISPA/IUCC)*, Dec. 2017, pp. 383–390.
- [19] A. Fert, V. Cros, and J. Sampaio, “Skyrmions on the track,” *Nat. Nanotechnol.*, vol. 8, no. 3, pp. 152–156, 2013.
- [20] X. Zhang, M. Ezawa, and Y. Zhou, “Magnetic skyrmion logic gates: Conversion, duplication and merging of skyrmions,” *Sci. Rep.*, vol. 5, no. 1, pp. 1–8, 2015.
- [21] X. Xing, P. W. Pong, and Y. Zhou, “Skyrmion domain wall collision and domain wall-gated skyrmion logic,” *Phys. Rev. B*, vol. 94, no. 5, p. 054408, 2016.
- [22] Y. C. Liaw, S. H. Chen, Y. H. Chang, and Y. P. Liang, “Sky-NN: Enabling efficient neural network data processing with skyrmion racetrack memory,” in *Proc. 2023 IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Aug. 2023, pp. 1–6.
- [23] X. Xin, Y. Zhang, and J. Yang, “ELP2IM: Efficient and low power bitwise operation processing in DRAM,” in *Proc. 2020 IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 303–314.
- [24] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, *et al.*, “Ambit: In-memory accelerator for bulk bitwise operations using commodity DRAM technology,” in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 273–287.
- [25] H. Li, H. Jin, L. Zheng, Y. Huang, X. Liao, Z. Duan, *et al.*, “ReSMA: Accelerating approximate string matching using ReRAM-based content addressable memory,” in *Proc. 59th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2022, pp. 991–996.
- [26] D. Niu, C. Xu, N. Muralimanohar, N. P. Jouppi, and Y. Xie, “Design of cross-point metal-oxide ReRAM emphasizing reliability and cost,” in *Proc. 2013 IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2013, pp. 17–23.

- [27] BWA-MEM2 ERT2022. [Online]. Available: <https://github.com/bwa-mem2/bwa-mem2/tree/ert>
- [28] Y. Huang, L. Kong, D. Chen, Z. Chen, X. Kong, J. Zhu, *et al.*, “Casa: An energy-efficient and high-speed CAM-based SMEM seeding accelerator for genome alignment,” in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2023, pp. 1423–1436.
- [29] L. Kull, T. Toifl, M. Schmatz, P. A. Francese, C. Menolfi, M. Braendli, *et al.*, “A 3.1 mW 8b 1.2 GS/s single-channel asynchronous SAR ADC with alternate comparators for enhanced speed in 32 nm digital SOI CMOS,” *IEEE J. Solid-State Circuits*, vol. 48, no. 12, pp. 3049–3058, 2013.

